Streaming Service Platform Database

- **Business/Organization Description:**

Overview of the Streaming Service

The streaming industry has transformed digital entertainment, allowing users to access a vast collection of movies, TV shows, and exclusive content anytime, anywhere. This project focuses on designing a robust and scalable database solution for an on-demand streaming service that manages users, subscriptions, content, streaming analytics, and personalized recommendations.
The service follows a subscription-based model, offering different tiers such as Basic, Standard, and Premium plans. Users can create multiple profiles under a single account, enabling personalized experiences for individuals. The platform supports content licensing, advertising-based monetization, and real-time analytics to optimize engagement.

Purpose and Benefits of Developing a Database

Managing large-scale streaming operations requires an efficient database solution to handle user interactions, content updates, and analytics. Implementing a well-structured database will provide several advantages. A well-structured database is essential to ensure **efficient data management, seamless content delivery, and insightful decision-making**. The database solution will provide several advantages, including:

1. **Efficient Subscriber Management**
    - Stores user accounts, profiles, authentication details, and subscription plans.
    - Manages multiple device logins with details like IP address and last login.
    - Automates payments, renewal tracking, and user access control.
2. **Comprehensive Content Cataloging**
    - Stores metadata for movies, TV series, and episodes, including title, genre, release date, ratings, and duration.
    - Tracks content licensing agreements with region-based validity periods.
    - Associates actors, directors, and content creators with relevant works.
3. **Streaming Analytics and Performance Tracking**
    - Logs user watch history, including timestamps, progress, and last-watched details.

    - Tracks advertisements viewed by users for targeted marketing.
    - Analyzes server load, peak viewing times, and engagement trends.
4. **Personalized Recommendations and Engagement**
    - Maintains a recommendation system based on watch history and user behavior.
    - Allows users to add content to a watchlist for future viewing.
    - Collects user reviews and ratings for content, enhancing content discovery.
5. **Customer Support and User Interaction**
    - Handles customer support tickets related to technical or billing issues.
    - Monitors user complaints and resolution statuses.

Functionalities and Features

The database solution will support key functionalities to improve operational efficiency and decision-making, including:

- **User & Profile Management** – Handles account creation, profile settings, authentication, and login tracking.
- **Content Library Management** – Organizes and categorizes content with metadata, licensing, and distribution details.
- **Subscription and Payment Processing** – Manages billing cycles, payment transactions, and pricing tiers.
- **Viewing History and Engagement Analytics** – Logs content interactions for improved recommendations and insights.
- **Personalized Recommendations** – Leverages user data to suggest relevant content dynamically.
- **Security and Compliance** – Ensures data protection, access control, and adherence to regulatory standards.
- **Advertisement Tracking** – Stores details on advertisements displayed, duration, target audience, and engagement.
- **Customer Support System** – Records user complaints and resolution statuses for better service.

By developing an **optimized and scalable MySQL database**, the streaming service can efficiently manage **millions of daily transactions**, ensure smooth content delivery, and provide a highly personalized user experience.

- **Conceptual Design – Enhanced Entity-Relationship (EER) Model**

The conceptual design of the streaming service database is structured around core entities that support user management, content cataloging, streaming analytics, advertisements, and personalized recommendations. The following section details the key entities, attributes, and relationships necessary for efficient business process management.

Key Entities and Their Attributes

1. User Management : Manages user accounts, multiple profiles, and login devices.

User : user_id (INT) **(PK),** name (VARCHAR(100)), email (VARCHAR(255)), password (VARCHAR(255)), country (VARCHAR(100)), date_of_birth (DATE), plan_id (INT) **(FK)** – Links to **Subscription Plans**

Profiles: profile_id (INT) **(PK),** user_id (INT) **(FK)** – Linked to **User,** profile_name (VARCHAR(50)), age_restriction (BOOLEAN), preferred_language (VARCHAR(50)), last_active_device (VARCHAR(100)

Devices: device_id (INT) **(PK),** user_id (INT) **(FK)** – Linked to **User,** device_type (VARCHAR(50)), os (VARCHAR(50)), ip_address (VARCHAR(50)), last_login (DATETIME)

2. Subscription and Payment Processing : Manages different subscription plans and payment transactions.

Subscription Plans (Superclass): plan_id (INT) **(PK),** plan_name (VARCHAR(50)), price (DECIMAL(10,2)), duration (INT), features (VARCHAR(255)), Type (ENUM('Basic', 'Standard', 'Premium'))

Basic Plan (Subclass): plan_id (INT) (PK, FK), ad_id (INT) (FK), ad_duration (INT)

Standard Plan (Subclass): plan_id (INT) (PK, FK), device_limit (INT), offline_download (BOOLEAN)

Premium Plan (Subclass): plan_id (INT) (PK, FK),Spatial_Audio_Support (BOOLEAN)

Payments: payment_id (INT) **(PK),** user_id (INT) **(FK)** – Linked to **User,** plan_id (INT) **(FK),**

amount (DECIMAL(10,2)), payment_date (DATE), payment_method (ENUM('Credit Card', 'PayPal', 'Bank Transfer'))

3. Content Management : Stores all media-related details, including metadata, episodes, and associated actors/directors.

Content: content_id (INT) **(PK),** title (VARCHAR(255)), genre (VARCHAR(50)), release_date (DATE), rating (INT), language (VARCHAR(50)), duration (INT)

Episodes: episode_id (INT) **(PK),** content_id (INT) **(FK),** season_number (INT), episode_number (INT), duration (INT)

Actors_Directors: person_id (INT) **(PK, FK)**, name (VARCHAR(100)), role (VARCHAR(50)), biography (TEXT)

4. Streaming Analytics and Personalization : Tracks user activity and generates personalized content suggestions.

Watch History: history_id (INT) **(PK),** profile_id (INT) **(FK) ,**content_id (INT) **(FK)**, watch_time (TIME), progress (VARCHAR(20))**,** last_watched (TIME)

Recommendations: recommendation_id (INT) **(PK),** profile_id (INT) **(FK),** content_id (INT) **(FK),** view_date (DATETIME)

5. User Engagement and Feedback : Users can interact with content through reviews, watchlists, and ratings.

Watchlist: profile_id (INT) **(PK, FK)**, content_id (INT) **(PK, FK)**, added_date (DATETIME)

Watch_History: history_id (INT) (PK), profile_id (INT) (FK), content_id (INT) (FK), watch_time (TIME), progress (VARCHAR(255)), last_watched (TIME)

Reviews: review_id (INT) **(PK)**, profile_id (INT) **(FK),** content_id (INT) **(FK),** rating (INT), comment (TEXT), review_date (VARCHAR(20))

6. Advertisement and Monetization : Handles targeted advertising for revenue generation.

Advertisement: AD_ID (INT) (**PK**), AD_name (VARCHAR(255)), AD_duration (INT), Category (VARCHAR(100)), Target_region (VARCHAR(100))

AD_views: ad_view_id (INT) (**PK**), user_id (INT) (**FK**), ad_id (INT) (**FK**), view_date (DATETIME)

7. Licensing and Compliance : Ensures legal streaming rights based on agreements.

Content_Licensing: license_id (INT) **(PK),** content_id (INT) **(FK),** region (VARCHAR(100)), start_date (DATE), end_date (DATE), license_holder (VARCHAR(255))

8. Customer Support : Manages user complaints and issues resolutions.

Customer Support : ticket_id (INT) **(PK),** user_id (INT) **(FK),** issue_type (ENUM('Technical', 'Billing', 'Account')), status (ENUM('Open', 'Resolved', 'Pending')), created_at (DATETIME), resolved_at (DATETIME)

Based on the provided ER diagram and the definition of relationships, here are the identified relationships along with their characteristics:

Relationships and Their Characteristics

1. **User "creates" Profiles : Degree:** Binary (User → Profile), **Cardinality:** One-to-Many (One user can create multiple profiles and each Profile belongs to one User)
2. **User "processes" Payments : Degree:** Binary (User → Payment), **Cardinality:** One-to-Many (One User can make multiple Payments, but each Payment belongs to a single User)
3. **User "signs in" on Devices : Degree:** Binary (User → Device), **Cardinality:** One-to-Many (A User can have multiple Subscription Plans, and a Subscription Plan can have multiple Users)
4. **User "files" Customer Support Tickets : Degree:** Binary (User → Customer Support), **Cardinality:** One-to-Many (One User can register multiple Devices, but each Device is associated with only one User)
5. **User "views" Advertisements : Degree:** Binary (User → AD_views), **Cardinality:** One-to-Many (One User can log multiple Watch History records, but each Watch History entry belongs to only one User)
6. **Payments "are associated with" Subscription Plans: Degree:** Binary (Payment → Subscription Plans), **Cardinality:** Many-to-One (Multiple Watch History records can refer to the same Content, but each Watch History entry is linked to only one Content)
7. **Profiles "logs" Watch History: Degree:** Binary (Profile → Watch_History), **Cardinality:** One-to-Many (A Recommendation can contain multiple Content items, and a Content item can be part of multiple Recommendations)

8. **Profiles "rates" Content (via Reviews) : Degree:** Binary (Profile → Reviews → Content), **Cardinality:** One-to-Many (Multiple Customer Support requests can be made by a User, but each request belongs to one User)
9. **Profiles "adds" Content to Watchlist: Degree:** Binary (Profile → Watchlist → Content), **Cardinality:** Many-to-Many (Profiles can add multiple content items, and content can appear in multiple watchlists)
10. **Profiles "receive" Recommendations: Degree:** Binary (Profile → Recommendations → Content), **Cardinality:** Many-to-Many (Profiles can receive multiple recommendations, and content can be recommended to multiple profiles)
11. **Content "contains" Episodes: Degree:** Binary (Content → Episodes), **Cardinality:** One-to-Many (One Content can include multiple Advertisements, but each Advertisement is linked to a single Content)
12. **Content "features" Actors/Directors: Degree:** Binary (Content → Actors_Directors), **Cardinality:** Many-to-Many (One Admin can manage multiple Content items, but each Content is managed by only one Admin)
13. **Content "is regulated by" Content Licensing: Degree:** Binary (Content → Content_Licensing), **Cardinality:** One-to-Many (One Content item can have multiple licensing/download rules, but each licensing rule applies to one Content)

Each of these relationships follows the defined business rules and cardinalities of the system represented in the ER diagram.

Associative Entity: An **associative entity** is a combination of both a relationship and an entity. It connects instances of one or more entity types while storing attributes that are specific to their association.

Associative Entities in the ER Diagram

1. **AD_Views**

   Key Attributes: ad_view_id (PK), user_id (FK), ad_id (FK), view_date.

   Purpose: Tracks advertisement impressions for each user, helping in analytics and targeted ad strategies.
2. **Recommendations**

   Key Attributes: recommendation_id (PK), profile_id (FK), content_id (FK), view_date.

Purpose: Facilitates personalized content suggestions based on user preferences and watch history.

3. **Watchlist**

   Key Attributes: profile_id (PK, FK), content_id (PK, FK), added_date.

   Purpose: Enables users to curate a list of content for future viewing, enhancing engagement and retention.

Supertypes and Subtypes

A supertype is a general entity that groups multiple subtypes, which are specialized entities with distinct attributes while inheriting common characteristics from the supertype.

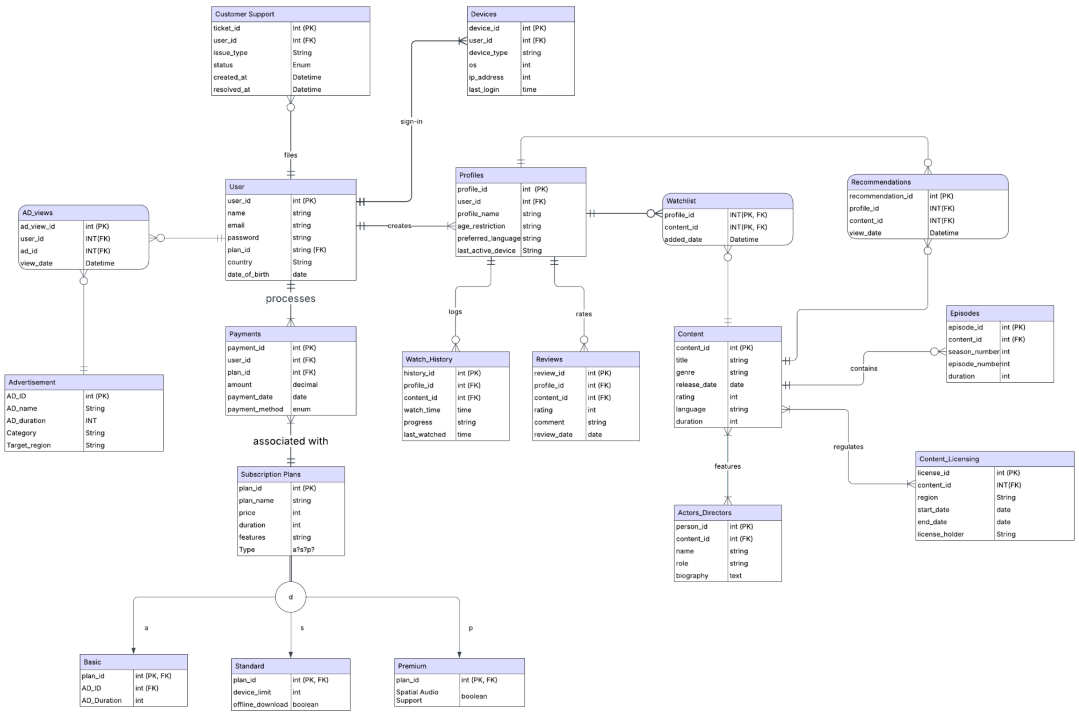
Supertype - Subscription Plans (generic entity for different plan types).

Subtypes - Basic, Standard, and Premium (each with unique attributes).

Total Specialization Rule (Double Line) - Every instance of the supertype must belong to at least one subtype. A Subscription Plan must be either Basic, Standard, or Premium.

Disjoint Rule (Only One Subtype) - An instance of the supertype can belong to only one subtype. A plan cannot be both Basic and Standard at the same time.

ER Model :



- **Logical Design**

Removing transitive dependencies and 3NF Normalization:

A **transitive dependency** occurs when a non-key attribute is functionally dependent on another non-key attribute, rather than directly on the primary key. In **Third Normal Form (3NF)**, all transitive dependencies must be removed by placing them into a separate table.

1. Actors_Directors Table

Issue: The attributes name and biography are functionally dependent on person_id, rather than directly on the composite key (person_id, content_id). **Solution:**

a. Create a new **Person** table with person_id as the primary key.
b. Store name and biography in the **Person** table.
c. Keep person_id and content_id in the **Actors_Directors** table along with role.
d. This eliminates transitive dependency and ensures **3NF compliance**.

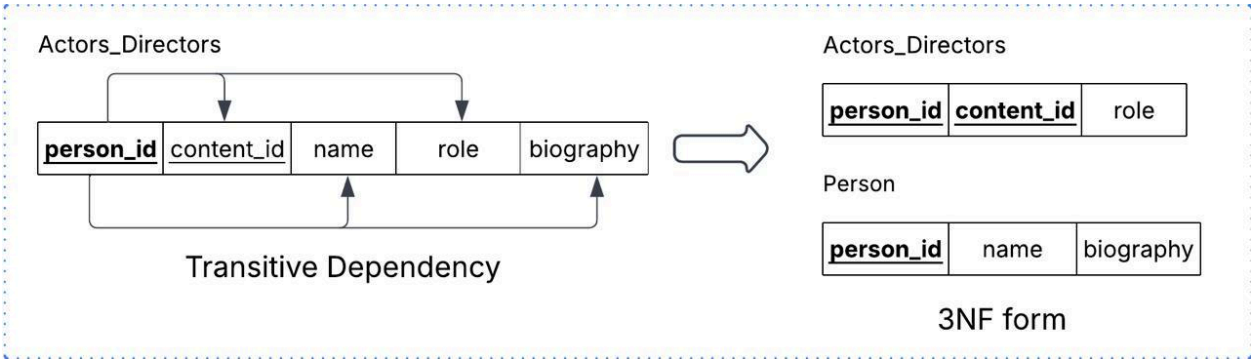


Fig: Elimination of transitive dependency in Actor_Directors table

2. Basic Plan Table

Issue: The attribute AD_Duration is functionally dependent on AD_ID, and AD_ID is a foreign key in the Basic table. This creates a transitive dependency. **Solution:**

a. Since AD_Duration belongs to the **Advertisement** table, it should not be stored in the Basic table.
b. Remove AD_Duration from Basic and keep only plan_id and AD_ID.
c. When required, AD_Duration can be fetched by joining with the Advertisement table using AD_ID.

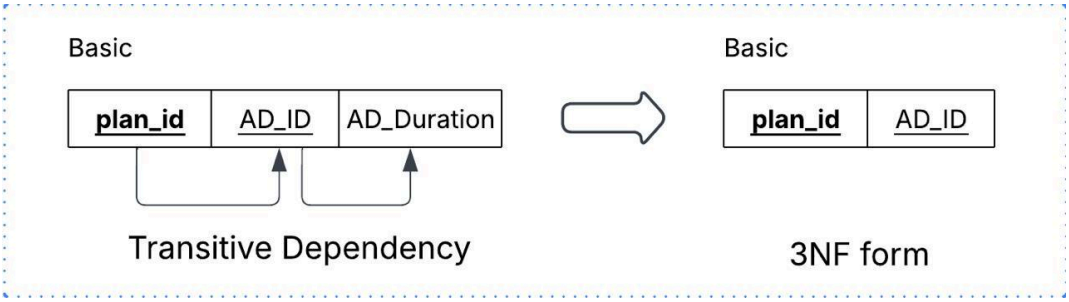
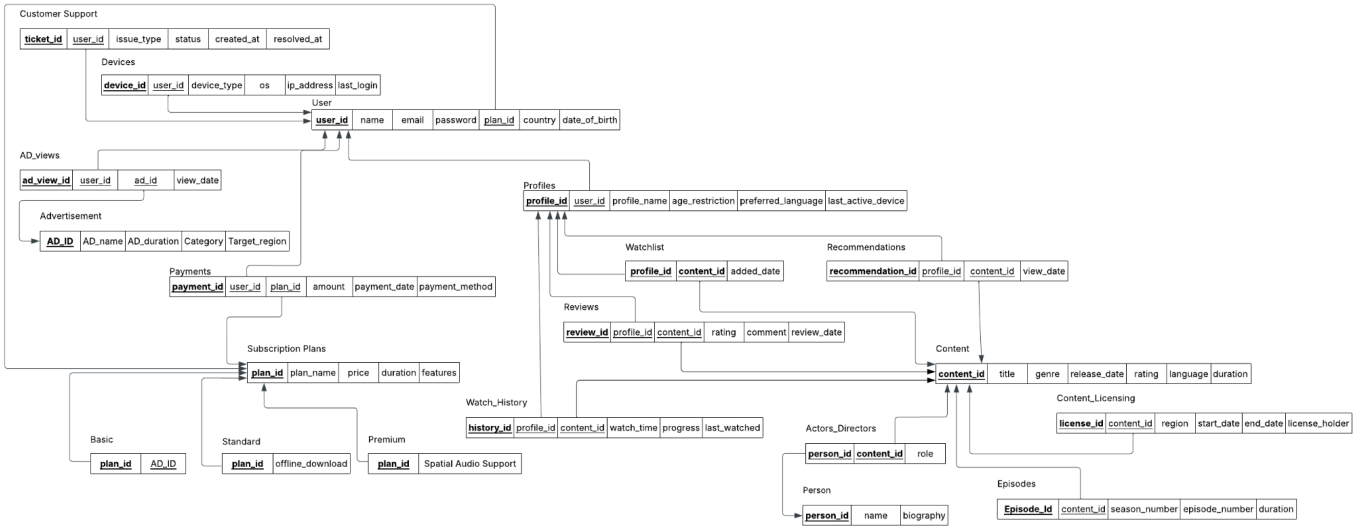


Fig: Elimination of transitive dependency in Basic_plan table

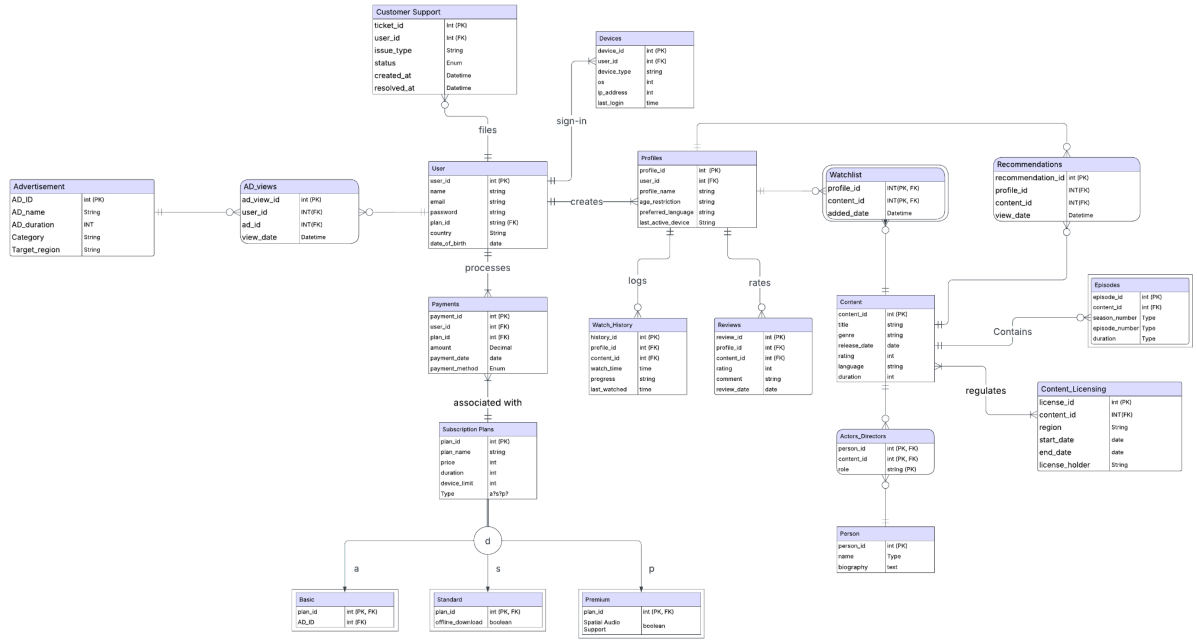
General Rule for 3NF: A **non-key determinant** (an attribute that determines another non-key attribute) should be placed in a new table. The non-key determinant **becomes the primary key** of the new table while remaining a **foreign key** in the original table. This ensures **data integrity, eliminates redundancy, and maintains efficient database structure**.

- **Relational Model:**

  A **Relational model** represents the **logical structure** of a database using relations (**tables**), attributes (**columns**), and tuples (**rows**). It is derived from an **Enhanced Entity-Relationship (EER) model**, ensuring that all **entities, relationships, and constraints** are mapped effectively into a structured format. To maintain **data integrity and eliminate redundancy**, the model is designed in **Third Normal Form (3NF)**



Final - ER diagram that satisfy 3 - Normal Form



▪ Implementation and Application:

• Provide some sample data for the major relations if needed.

```
CREATE TABLE IF NOT EXISTS `User` (
    user_id INT PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    email VARCHAR(255) UNIQUE NOT NULL,
    password VARCHAR(255) NOT NULL,
    plan_id INT,
    country VARCHAR(100),
    date_of_birth DATE,
    FOREIGN KEY (plan_id) REFERENCES Subscription_Plans(plan_id)
    ON DELETE CASCADE ON UPDATE CASCADE
);

INSERT INTO `User` (user_id, name, email, password, plan_id, country, date_of_birth)
VALUES
(1, 'Alice Johnson', 'alice1@example.com', 'hashed_password1', 1, 'USA', '1990-05-14'),
(2, 'Bob Smith', 'bob2@example.com', 'hashed_password2', 3, 'UK', '1985-10-20'),
```

user_id	name	email	password	plan_id	country	date_of_bir...
1	Alice Johnson	alice1@example.com	hashed_password1	1	USA	1990-05-14
2	Bob Smith	bob2@example.com	hashed_password2	3	UK	1985-10-20
3	Charlie Davis	charlie3@example.com	hashed_password3	3	Canada	1992-07-01
4	Daniel Martinez	daniel4@example.com	hashed_password4	2	Spain	1995-06-23
5	Emily White	emily5@example.com	hashed_password5	2	France	1988-11-12
6	Frank Harris	frank6@example.com	hashed_password6	3	Germany	1993-09-15
7	Grace Lee	grace7@example.com	hashed_password7	1	South Korea	1991-03-28
8	Henry Kim	henry8@example.com	hashed_password8	1	Japan	1987-08-17
9	Isabella Brown	isabella9@example.com	hashed_password9	3	Italy	1994-02-04
10	Jack Wilson	jack10@example.com	hashed_password10	2	Australia	1996-12-09
11	Karen Thomp...	karen11@example.com	hashed_password11	2	India	1989-07-21
12	Leo Walker	leo12@example.com	hashed_password12	3	Brazil	1997-01-30
13	Mia Scott	mia13@example.com	hashed_password13	1	Russia	1992-05-11
14	Nathan Moore	nathan14@example.com	hashed_password14	1	UAE	1986-04-22
15	Olivia King	olivia15@example.com	hashed_password15	2	South Africa	1995-10-05

Basic Queries

```sql
SELECT c.title, COUNT(r.review_id) AS total_reviews
FROM Reviews r
JOIN Content c ON r.content_id = c.content_id
GROUP BY c.title
ORDER BY total_reviews DESC
LIMIT 10;
```

```sql
SELECT p.profile_id, p.profile_name, c.title, wh.progress
FROM Watch_History wh
JOIN Profiles p ON wh.profile_id = p.profile_id
JOIN Content c ON wh.content_id = c.content_id
ORDER BY p.profile_id, wh.progress DESC;
```

Analysis

1. Top & Bottom Performing Genres by Watch Hours Across Countries

Querry:

```
WITH WatchTimePerCountry AS (
    SELECT u.country, c.genre,
```

```
        SUM(TIME_TO_SEC(wh.watch_time)/3600) AS total_watch_hours
    FROM Watch_History wh
    JOIN Profiles p ON wh.profile_id = p.profile_id
    JOIN  User  u ON p.user_id = u.user_id
    JOIN Content c ON wh.content_id = c.content_id
    GROUP BY u.country, c.genre
),
RecommendationsPerCountry AS (
    SELECT u.country, c.genre, COUNT(r.recommendation_id) AS total_recommendations
    FROM Recommendations r
    JOIN Profiles p ON r.profile_id = p.profile_id
    JOIN  User  u ON p.user_id = u.user_id
    JOIN Content c ON r.content_id = c.content_id
    GROUP BY u.country, c.genre
),
RankedWatchTime AS (
    SELECT w.country, w.genre, w.total_watch_hours,
        COALESCE(r.total_recommendations, 0) AS total_recommendations
    FROM WatchTimePerCountry w
    LEFT JOIN RecommendationsPerCountry r
    ON w.country = r.country AND w.genre = r.genre
)
(
    SELECT * FROM RankedWatchTime
    ORDER BY total_watch_hours DESC
    LIMIT 3
)
UNION ALL
(
    SELECT * FROM RankedWatchTime
    ORDER BY total_watch_hours ASC
    LIMIT 3
);
```

country	genre	total_watch_hours	total_recommendations
South Africa	Sports	26.4167	0
India	Western	19.9166	0
Spain	Thriller	15.5000	0
Canada	Sci-Fi	0.1667	1
Italy	Romance	0.6667	2
UK	Drama	0.7500	2

2. Predicting Customer Retention Based on Support Resolution Time

```
WITH ResolutionTime AS (
   SELECT cs.user_id,
        AVG(TIMESTAMPDIFF(HOUR, cs.created_at, cs.resolved_at)) AS avg_resolution_time,
        COUNT(cs.ticket_id) AS total_issues
   FROM Customer_Support cs
   WHERE cs.status = 'Resolved'
   GROUP BY cs.user_id
),
PaymentActivity AS (
   SELECT u.user_id, COUNT(p.payment_id) AS total_payments
   FROM Payments p
   JOIN  User  u ON p.user_id = u.user_id
   GROUP BY u.user_id
)
SELECT r.user_id, u.name, u.country, sp.plan_name,
      r.total_issues, r.avg_resolution_time,
      COALESCE(pa.total_payments, 0) AS payment_count
FROM ResolutionTime r
JOIN  User  u ON r.user_id = u.user_id
JOIN Subscription_Plans sp ON u.plan_id = sp.plan_id
LEFT JOIN PaymentActivity pa ON r.user_id = pa.user_id
WHERE r.avg_resolution_time > 48
  AND COALESCE(pa.total_payments, 0) < 2
ORDER BY avg_resolution_time DESC;
```

user_id	name	country	plan_name	total_issues	avg_resolution_time	payment_count
1	Alice Johnson	USA	Basic	1	50.0000	1

3. Calculate total revenue generated by users who watched ads in each category along with the peak hours

```
WITH AdRevenue AS (
   SELECT a.category,
        SUM(p.amount) AS total_revenue,
        COUNT(av.ad_view_id) AS total_ad_views
   FROM AD_views av
   JOIN Advertisement a ON av.ad_id = a.ad_id
   JOIN Payments p ON av.user_id = p.user_id
   GROUP BY a.category
```

```
),
PeakAdHours AS (
    SELECT a.category,
           HOUR(av.view_date) AS peak_hour
    FROM AD_views av
    JOIN Advertisement a ON av.ad_id = a.ad_id
    GROUP BY a.category, peak_hour
    ORDER BY a.category, COUNT(av.ad_view_id) DESC
)
SELECT ar.category,
       ar.total_revenue,
       ar.total_ad_views,
       pa.peak_hour
FROM AdRevenue ar
JOIN PeakAdHours pa ON ar.category = pa.category
WHERE pa.peak_hour = (
    SELECT HOUR(av.view_date)
    FROM AD_views av
    JOIN Advertisement a ON av.ad_id = a.ad_id
    WHERE a.category = pa.category
    GROUP BY HOUR(av.view_date)
    ORDER BY COUNT(av.ad_view_id) DESC
    LIMIT 1
)
ORDER BY ar.total_revenue DESC;
```

category	total_revenue	total_ad_views	peak_hour
Entertainment	30.00	2	15
Food	20.00	2	12
Technology	10.00	2	14
Finance	15.00	1	13
Healthcare	15.00	1	14
Automobile	15.00	1	20
Sports	10.00	1	18
Education	10.00	1	17
Real Estate	10.00	1	11
Lifestyle	5.00	1	9
Travel	5.00	1	21
Fitness	5.00	1	9

Conclusion

This project successfully designed a scalable and efficient database for a subscription-based streaming service. The system supports user management, content cataloging, streaming analytics, personalized recommendations, and customer support while ensuring data integrity and minimal redundancy.

Optimized SQL queries provide insights into content performance, customer retention, and ad revenue tracking. By leveraging MySQL and structured data modeling, the database enhances user engagement and operational efficiency. Future improvements could include machine learning for advanced recommendations and predictive analytics, further optimizing the streaming experience.

Project by, Group No: 9

Group members :

1. Ankita Dhekane : adhekane@scu.edu
2. Maheshwari Bhandare : mbhandare@scu.edu
3. Shwetalee Gadekar : sgadekar@scu.edu
4. Trupti Lone : tlone@gmail.com